

goat

Sunday, November 10, 2024 12:53 PM

Must Run this on ALL terminals for it to work

```
cd ~/Desktop/luna
colcon build --symlink-install
source install/setup.bash
```

- 1 ros2 launch realsense2_camera rs_launch.py
- 2 ros2 run open_cv_pkg filtering
- 3 ros2 run open_cv_pkg og

ghp_auyk5bl2Dd8lgLTebMjUHW2Inonqqe1Rmmpu

MonkeyNuts17!
Matthewe17!
Gatesfour44!

ros2 launch pointcloud_to_grid rviz.launch.py

ros2 run open_cv_pkg object_detection

ros2 param set /pointcloud_to_grid_node cloud_in_topic /camera/camera/depth/color/points
Had to set in pointcloud to grid

Depth_module.hdr enables usually false

Just depth

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2
from nav_msgs.msg import OccupancyGrid
from std_msgs.msg import Header
import sensor_msgs_py.point_cloud2 as pc2
import numpy as np
class OccupancyGridFromPointCloud(Node):
    def __init__(self):
        super().__init__('occupancy_grid_from_pointcloud')

        # Publisher to publish the occupancy grid
        self.pub = self.create_publisher(OccupancyGrid, 'occupancy_grid', 10)
        # Subscribe to the point cloud topic
        self.sub = self.create_subscription(
            PointCloud2,
            '/camera/camera/depth/color/points', # Change topic if needed
            self.pointcloud_callback,
            10
        )

        self.get_logger().info('Occupancy Grid Node started.')
        # Define grid parameters
        self.grid_size = 0.1 # The resolution of the grid (meters per cell)
        self.grid_width = 100 # Number of grid cells in width
        self.grid_height = 100 # Number of grid cells in height
        # Initialize the occupancy grid with zeros (empty grid)
        self.occupancy_grid = np.zeros((self.grid_height, self.grid_width))
    def pointcloud_callback(self, msg):
        """Process point cloud data and update the occupancy grid."""

        # Extract the point cloud data from the ROS message
        point_list = list(pc2.read_points(msg, field_names=('x', 'y', 'z'), skip_nans=True))
        points = np.array(point_list, dtype=[('x', 'f4'), ('y', 'f4'), ('z', 'f4')])
        # Log the point cloud data size for debugging
        self.get_logger().info(f"Point cloud data received with {len(points)} points.")

        # Filter out points that are too far away or not within the expected depth range
        filtered_points = self.filter_boulders_and_craters(points)
        self.get_logger().info(f"Filtered points: {len(filtered_points)} points.") # Log filtered points
        if len(filtered_points) == 0:
            self.get_logger().warn("No points left after filtering.") # Warn if no points passed filtering

        # Convert filtered point cloud to 2D grid coordinates
        for point in filtered_points:
            x, y, z = point
            # Convert to grid cell coordinates (assuming the center of the robot is at the origin)
            grid_x = int((x + self.grid_size * self.grid_width / 2) // self.grid_size)
            grid_y = int((y + self.grid_size * self.grid_height / 2) // self.grid_size)
            # Ensure the point falls within the grid's boundaries
            if 0 <= grid_x < self.grid_width and 0 <= grid_y < self.grid_height:
                # Mark the grid cell as occupied (obstacle)
                self.occupancy_grid[grid_y, grid_x] = 100 # Occupied cell
                self.get_logger().info(f"Mapped point ({x}, {y}, {z}) to grid position ({grid_x}, {grid_y})")
        # Log the number of occupied cells
        occupied_cells = np.count_nonzero(self.occupancy_grid)
        self.get_logger().info(f"Number of occupied cells: {occupied_cells}")
        if occupied_cells == 0:
            self.get_logger().warn("No occupied cells detected in the occupancy grid.")
        # Create an OccupancyGrid message
        grid_msg = OccupancyGrid()
        grid_msg.header = Header()
        grid_msg.header.stamp = self.get_clock().now().to_msg()
        grid_msg.header.frame_id = "camera_link" # Set the frame of reference (could also use robot base frame)

        grid_msg.info.resolution = self.grid_size # The resolution of each grid cell
        grid_msg.info.width = self.grid_width
        grid_msg.info.height = self.grid_height

        # Flatten the occupancy grid array and convert it to a list of integers
```

```

grid_msg.data = self.occupancy_grid.flatten().astype(int).tolist()

# Publish the occupancy grid
self.pub.publish(grid_msg)
self.get_logger().info(f"Published Occupancy Grid with {occupied_cells} occupied cells.")
def filter_boulders_and_craters(self, points):
    """Filter out irrelevant points, keeping only boulders and craters based on depth and size."""

    # Depth threshold values (adjust based on your environment)
    min_depth = 0.1 # Min depth (close objects like boulders/craters)
    max_depth = 2.0 # Max depth (objects far away like walls)
    # Extract the z-axis (depth) values and filter based on depth range
    z_values = points['z']
    filtered_points = points[(z_values >= min_depth) & (z_values <= max_depth)]

    # Log the number of points after filtering
    self.get_logger().info(f"Filtered points based on depth: {len(filtered_points)}")

    return filtered_points
def main(args=None):
    rclpy.init(args=args)
    node = OccupancyGridFromPointCloud()

    try:
        rclpy.spin(node) # Keep the node running and processing incoming data
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node() # Gracefully shut down the node
        rclpy.shutdown()
if __name__ == '__main__':
    main()

```

bruh

```

// ROS
#include "rclcpp/rclcpp.hpp"
#include "nav_msgs/msg/occupancy_grid.hpp"
#include "sensor_msgs/msg/point_cloud.hpp"
#include "sensor_msgs/msg/point_cloud2.hpp"
#include "rcl_interfaces/msg/set_parameters_result.hpp"
// PCL
#include <pcl/point_cloud.h>
#include <pcl/point_types.h>
#include <pcl_conversions/pcl_conversions.h>
#include <pcl/filters/crop_box.h>
// ROS package
#include <pointcloud_to_grid/pointcloud_to_grid_core.hpp>
// C++
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include <cmath>
#include <vector>
#include <algorithm>
// nav_msgs::msg::OccupancyGrid: Ptr height_grid(new nav_msgs::msg::OccupancyGrid);
auto height_grid = std::make_shared<nav_msgs::msg::OccupancyGrid>();
auto intensity_grid = std::make_shared<nav_msgs::msg::OccupancyGrid>();
class PointCloudToGrid : public rclcpp::Node
{
    rcl_interfaces::msg::SetParametersResult parametersCallback(const
std::vector<rclcpp::Parameter> &parameters)
    {
        rcl_interfaces::msg::SetParametersResult result;
        result.successful = true;
        result.reason = "success";
        for (const auto &param : parameters)
        {
            RCLCPP_INFO_STREAM(this->get_logger(), "Param update: " << param.get_name().c_str()
<< ": " << param.value_to_string().c_str());
            if (param.get_name() == "mapi_topic_name")
            {
                grid_map.mapi_topic_name = param.as_string();
                pub_igrd = this->create_publisher<nav_msgs::msg::OccupancyGrid>
(grid_map.mapi_topic_name, 10);
            }
        }
    }
}

```

Temp 803

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2
from nav_msgs.msg import OccupancyGrid
from std_msgs.msg import Header
import sensor_msgs_py.point_cloud2 as pc2
import numpy as np
class OccupancyGridFromPointCloud(Node):
    def __init__(self):
        super().__init__('occupancy_grid_from_pointcloud')

        # Publisher to publish the occupancy grid
        self.pub = self.create_publisher(OccupancyGrid,
'occupancy_grid', 10)
        # Subscribe to the point cloud topic
        self.sub = self.create_subscription(
PointCloud2,
'/camera/camera/depth/color/points', # Change topic if
needed
self.pointcloud_callback,
10
)

        self.get_logger().info('Occupancy Grid Node started.')
        # Define grid parameters
        self.grid_size = 0.1 # The resolution of the grid (meters per
cell)
        self.grid_width = 100 # Number of grid cells in width
        self.grid_height = 100 # Number of grid cells in height
        # Initialize the occupancy grid with zeros (empty grid)
        self.occupancy_grid = np.zeros((self.grid_height,
self.grid_width))
        def pointcloud_callback(self, msg):
            """Process point cloud data and update the occupancy
grid."""

            # Extract the point cloud data from the ROS message
            point_list = list(pc2.read_points(msg, field_names=("x", "y",
"z"), skip_nans=True))
            points = np.array(point_list, dtype=[('x', 'f4'), ('y', 'f4'), ('z',
'f4')])

            # Log the point cloud data size for debugging
            self.get_logger().info(f"Point cloud data received with
{len(points)} points.")

            # Filter out points that are too far away or not within the
expected depth range

```

```

        if (param.get_name() == "mapi_topic_name")
        {
            grid_map.mapi_topic_name = param.as_string();
            pub_lgrid = this->create_publisher<nav_msgs::msg::OccupancyGrid>(
grid_map.mapi_topic_name, 10);
        }
        if (param.get_name() == "maph_topic_name")
        {
            grid_map.maph_topic_name = param.as_string();
            pub_hgrid = this->create_publisher<nav_msgs::msg::OccupancyGrid>(
grid_map.maph_topic_name, 10);
        }
        if (param.get_name() == "cloud_in_topic")
        {
            cloud_in_topic = param.as_string();
            sub_pc2_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(cloud_in_topic, 10,
std::bind(&PointCloudToGrid::lidar_callback, this, std::placeholders::_1));
        }
        if (param.get_name() == "cell_size")
        {
            grid_map.cell_size = param.as_double();
        }
        if (param.get_name() == "position_x")
        {
            grid_map.position_x = param.as_double();
        }
        if (param.get_name() == "position_y")
        {
            grid_map.position_y = param.as_double();
        }
        if (param.get_name() == "length_x")
        {
            grid_map.length_x = param.as_double();
        }
        if (param.get_name() == "length_y")
        {
            grid_map.length_y = param.as_double();
        }
        if (param.get_name() == "intensity_factor")
        {
            grid_map.intensity_factor = param.as_double();
        }
        if (param.get_name() == "height_factor")
        {
            grid_map.height_factor = param.as_double();
        }
        if (param.get_name() == "verbose1")
        {
            verbose1 = param.as_bool();
        }
        if (param.get_name() == "verbose2")
        {
            verbose2 = param.as_bool();
        }
        // grid_map.frame_out = config.frame_out;
        grid_map.paramRefresh();
    }
    return result;
}
public:
PointCloudToGrid() : Node("pointcloud_to_grid_node"), count_(0)
{
    this->declare_parameter<std::string>("mapi_topic_name", "intensity_grid");
    this->declare_parameter<std::string>("maph_topic_name", "height_grid");
    this->declare_parameter<std::string>("cloud_in_topic", "/camera/camera/depth/color/points");
    this->declare_parameter<float>("cell_size", 0.5);
    this->declare_parameter<float>("position_x", 0.0);
    this->declare_parameter<float>("position_y", 0.0);
    this->declare_parameter<float>("length_x", 20.0);
    this->declare_parameter<float>("length_y", 30.0);
    this->declare_parameter<float>("intensity_factor", 1.0);
    this->declare_parameter<float>("height_factor", 1.0);
    this->declare_parameter<bool>("verbose1", verbose1);
    this->declare_parameter<bool>("verbose2", verbose2);
    this->get_parameter("mapi_topic_name", grid_map.mapi_topic_name);
    this->get_parameter("maph_topic_name", grid_map.maph_topic_name);
    this->get_parameter("cloud_in_topic", cloud_in_topic);
    this->get_parameter("cell_size", grid_map.cell_size);
    this->get_parameter("position_x", grid_map.position_x);
    this->get_parameter("position_y", grid_map.position_y);
    this->get_parameter("length_x", grid_map.length_x);
    this->get_parameter("length_y", grid_map.length_y);
    this->get_parameter("intensity_factor", grid_map.intensity_factor);
    this->get_parameter("height_factor", grid_map.height_factor);
    this->get_parameter("verbose1", verbose1);
    this->get_parameter("verbose2", verbose2);
    grid_map.paramRefresh();
    pub_lgrid = this->create_publisher<nav_msgs::msg::OccupancyGrid>(
grid_map.mapi_topic_name, 10);
    pub_hgrid = this->create_publisher<nav_msgs::msg::OccupancyGrid>(
grid_map.maph_topic_name, 10);
    sub_pc2_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(cloud_in_topic, 10,
std::bind(&PointCloudToGrid::lidar_callback, this, std::placeholders::_1));
    callback_handle_ = this->
add_on_set_parameters_callback(std::bind(&PointCloudToGrid::parametersCallback, this,
std::placeholders::_1));
    RCLCPP_INFO_STREAM(this->get_logger(), "pointcloud_to_grid_node has been started.");
    RCLCPP_INFO_STREAM(this->get_logger(), "Subscribing to: " << cloud_in_topic.c_str());
    RCLCPP_INFO_STREAM(this->get_logger(), "Publishing to: "
<< grid_map.mapi_topic_name.c_str() << " and " << grid_map.maph_topic_name.c_str());
}
private:
void
lidar_callback(const sensor_msgs::msg::PointCloud2::ConstSharedPtr input_msg)
{
    pcl::PointCloud<pcl::PointXYZI>::Ptr out_cloud(new pcl::PointCloud<pcl::PointXYZI>);
    pcl::fromROSMsg(*input_msg, *out_cloud);
    // Initialize grid
    grid_map.initGrid(intensity_grid);
    grid_map.initGrid(height_grid);
    // width*height/cell_size^2 eg width = 20m, height = 30m, cell_size data size = 6000
    // or cell_num_x * cell_num_ys
    // -128 127 int8[] data
    std::vector<signed char> hpoints(grid_map.cell_num_x * grid_map.cell_num_y);
    std::vector<signed char> ipoints(grid_map.cell_num_x * grid_map.cell_num_y);
    // initialize grid vectors: -128

```

```

    """
    # Log the point cloud data size for debugging
    self.get_logger().info(f"Point cloud data received with
{len(points)} points.")

    # Filter out points that are too far away or not within the
expected depth range
    filtered_points = self.filter_boulders_and_craters(points)
    self.get_logger().info(f"Filtered points: {len(filtered_points)}
points.") # Log filtered points
    if len(filtered_points) == 0:
        self.get_logger().warn("No points left after filtering.") #
Warn if no points passed filtering

    # Convert filtered point cloud to 2D grid coordinates
    for point in filtered_points:
        x, y, z = point
        # Convert to grid cell coordinates (assuming the center of
the robot is at the origin)
        grid_x = int((x + self.grid_size * self.grid_width / 2) //
self.grid_size)
        grid_y = int((y + self.grid_size * self.grid_height / 2) //
self.grid_size)
        # Ensure the point falls within the grid's boundaries
        if 0 <= grid_x < self.grid_width and 0 <= grid_y
< self.grid_height:
            # Mark the grid cell as occupied (obstacle)
            self.occupancy_grid[grid_y, grid_x] = 100 # Occupied
cell
            self.get_logger().info(f"Mapped point ({x}, {y}, {z}) to grid
position ({grid_x}, {grid_y})")
            # Log the number of occupied cells
            occupied_cells = np.count_nonzero(self.occupancy_grid)
            self.get_logger().info(f"Number of occupied cells:
{occupied_cells}")
            if occupied_cells == 0:
                self.get_logger().warn("No occupied cells detected in the
occupancy grid.")
            # Create an OccupancyGrid message
            grid_msg = OccupancyGrid()
            grid_msg.header = Header()
            grid_msg.header.stamp = self.get_clock().now().to_msg()
            grid_msg.header.frame_id = "camera_link" # Set the frame
of reference (could also use robot base frame)

            grid_msg.info.resolution = self.grid_size # The resolution of
each grid cell
            grid_msg.info.width = self.grid_width
            grid_msg.info.height = self.grid_height

            # Flatten the occupancy grid array and convert it to a list of
integers
            grid_msg.data =
self.occupancy_grid.flatten().astype(int).tolist()

            # Publish the occupancy grid
            self.pub.publish(grid_msg)
            self.get_logger().info(f"Published Occupancy Grid with
{occupied_cells} occupied cells.")
            def filter_boulders_and_craters(self, points):
                """Filter out irrelevant points, keeping only boulders and
craters based on depth and size."""

            # Depth threshold values (adjust based on your
environment)
            min_depth = 0.1 # Min depth (close objects like
boulders/craters)
            max_depth = 2.0 # Max depth (objects far away like walls)
            # Extract the z-axis (depth) values and filter based on depth
range
            z_values = points['z']
            filtered_points = points[(z_values >= min_depth) & (z_values
<= max_depth)]

            # Log the number of points after filtering
            self.get_logger().info(f"Filtered points based on depth:
{len(filtered_points)}")

            return filtered_points
def main(args=None):
    rclpy.init(args=args)
    node = OccupancyGridFromPointCloud()

    try:
        rclpy.spin(node) # Keep the node running and processing
incoming data
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node() # Gracefully shut down the node
        rclpy.shutdown()
if __name__ == '__main__':
    main()

```

```

for (auto &p : hpoints)
{
    p = -128;
}
for (auto &p : ipoints)
{
    p = -128;
}
for (pcl::PointXYZI p : out_cloud->points)
{
    if (p.x > 0.01 || p.x < -0.01)
    {
        if (p.x > grid_map.bottomright_x && p.x < grid_map.topleft_x)
        {
            if (p.y > grid_map.bottomright_y && p.y < grid_map.topleft_y)
            {
                PointXY cell = grid_map.getIndex(p.x, p.y);
                if (cell.x < grid_map.cell_num_x && cell.y < grid_map.cell_num_y)
                {
                    ipoints[cell.y * grid_map.cell_num_x + cell.x] = p.intensity * grid_map.intensity_factor;
                    hpoints[cell.y * grid_map.cell_num_x + cell.x] = p.z * grid_map.height_factor;
                }
            }
            else
            {
                RCLCPP_WARN_STREAM(this->get_logger(), "Cell out of range: " << cell.x << " - "
<< grid_map.cell_num_x << " ||| " << cell.y << " - " << grid_map.cell_num_y);
            }
        }
    }
}
}

intensity_grid->header.stamp = this->now();
intensity_grid->header.frame_id = input_msg->header.frame_id;
intensity_grid->info.map_load_time = this->now();
intensity_grid->data = ipoints;
height_grid->header.stamp = this->now();
height_grid->header.frame_id = input_msg->header.frame_id;
height_grid->info.map_load_time = this->now();
height_grid->data = hpoints;
pub_hgrid->publish(*height_grid);
pub_igrd->publish(*intensity_grid);
// pub_hgrid->publish(height_grid);
if (verbose1){
    RCLCPP_INFO_STREAM(this->get_logger(), "Published "
<< grid_map.mapi_topic_name.c_str() << " and " << grid_map.maph_topic_name.c_str());
}
}

rclcpp::Publisher<nav_msgs::msg::OccupancyGrid>::SharedPtr pub_igrd, pub_hgrid;
rclcpp::Subscription<sensor_msgs::msg::PointCloud2>::SharedPtr sub_pc2_;
OnSetParametersCallbackHandle::SharedPtr callback_handle_;
std::string cloud_in_topic = "nonground";
bool verbose1 = true, verbose2 = false;
// nav_msgs::msg::OccupancyGrid::Ptr intensity_grid =
std::make_shared<nav_msgs::msg::OccupancyGrid>();
// nav_msgs::msg::OccupancyGrid::Ptr intensity_grid(new nav_msgs::msg::OccupancyGrid);
// nav_msgs::msg::OccupancyGrid::Ptr height_grid =
std::make_shared<nav_msgs::msg::OccupancyGrid>();
GridMap grid_map;
size_t count_;
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<PointCloudToGrid>());
    rclcpp::shutdown();
    return 0;
}

```